



A

B. Tech (Information Technology)

PROJECT REPORT ON

**“DreamOS: An AI-Native Semantic File Management System with
Knowledge Graph, Context Memory and Intelligent Workspace
Management”**

Submitted by

Zaid Ali Syed 2305139

Om Vyas 2305170

Krushna Kadam 2305165

Under Guidance of

Dr. Minakshi R. Rajput

Institute of Information and Communication Technology (IICT)

MGM University, Chh. Sambhajinagar

Academic Year 2026-2027



Institute of Information and Communication Technology
MGM University, Chh. Sambhajinagar

CERTIFICATE

This is to certify that Zaid Ali Syed (Roll No. 2305139), Om Vyas (Roll No. 2305170) and Krushna Kadam (Roll No. 2305165) of B.Tech (IT), have successfully completed project work on “DreamOS: An AI-Native Semantic File Management System” under the guidance of Dr. Minakshi R. Rajput, and submitted the same during the academic year 2026-2027 towards the partial fulfillment of the degree of B.Tech (IT) from Institute of Information and Communication Technology, MGM University, Chh. Sambhajinagar.

Dr. Minakshi R. Rajput
Guide

Dr. S. C. Tamane
Director, IICT

Date: _____

Acknowledgement

We thank our guide, Dr. Minakshi R. Rajput, for guidance on the project. We thank Dr. S. C. Tamane, Director, IICT, and the faculty of the Institute of Information and Communication Technology for the environment and the Project Monitoring structure in which this work was reviewed.

We are grateful to the open-source communities behind Ollama, ChromaDB, FastAPI, Tauri, React and pytest, on whose work DreamOS is built, and to our families for their patience during the long build.

(Zaid Ali Syed, Roll No. 2305139)

(Om Vyas, Roll No. 2305170)

(Krushna Kadam, Roll No. 2305165)

Contents

Acknowledgement	ii
Abstract	v
Abbreviations	vi
List of Figures	vii
List of Tables	vii
Chapter 1: Introduction	1
1.1 Introduction	1
1.2 Problem Statement	1
1.3 Objectives	1
1.4 Scope of the Project	2
Chapter 2: Literature Review	3
2.1 Review of Existing Systems and Related Work	3
2.2 Research Gap	4
2.3 Summary of Findings	4
Chapter 3: System Analysis	5
3.1 Requirement Analysis	5
3.1.1 Functional requirements	5
3.1.2 Non-functional requirements	5
3.2 Feasibility Study	6
3.3 System Specifications	6
Chapter 4: System Design	7
4.1 System Architecture	7
4.2 Module Design	7
4.3 Data Flow Diagrams	8
4.4 UML Diagrams	8
4.5 Database Design	10
4.6 UI / UX Design	11
4.7 Technology Stack	11
Chapter 5: Implementation	12
5.1 Module-wise Description	12
5.1.1 Natural Language Interface	12
5.1.2 Semantic Search Engine	12
5.1.3 AI File Organiser	12
5.1.4 Knowledge Graph	12
5.1.5 Context Memory Engine	12

5.1.6 Intelligent Workspace Manager	13
5.1.7 Privacy audit	13
5.2 Technologies Used.....	13
5.3 Code Snippets	13
5.4 Screenshots of Outputs	14
Chapter 6: Testing and Validation	15
6.1 Testing Types.....	15
6.2 Test Cases and Results.....	15
Chapter 7: Results and Discussion.....	17
7.1 Performance Evaluation.....	17
7.2 Comparison with Existing Systems	18
7.3 Observations	18
Chapter 8: Conclusion and Future Scope.....	20
8.1 Summary of Work.....	20
8.2 Limitations	20
8.3 Future Enhancement	20
References.....	22
Appendices.....	23
Appendix A: Source code	23
Appendix B: Scale benchmark raw results	23
Appendix C: Papers and certificates	23

Abstract

Personal computers hold thousands of files whose names were chosen once and rarely describe what the file is about now. Finding a file means remembering how it was named. DreamOS is an AI-native desktop file manager that lets a user describe a file in plain language, and it finds, opens, relates and organises files by meaning rather than by name. All processing runs on the user's computer: embeddings and language understanding come from a local model runtime, and the index is stored in local databases, so no file content or question is sent to a server.

The system has six modules: a natural-language interface that classifies intent, a semantic search engine, an AI file organiser with reversible moves, a knowledge graph of relationships between files, a context memory engine that resolves follow-up questions, and an intelligent workspace manager that recommends and creates groups of related files. A privacy audit reports every network destination the software uses, so the claim that data stays local can be checked rather than trusted.

The work was evaluated on synthetic vaults of 500, 2,000 and 5,000 files. Indexing ran at 19.4 files per second at 500 files, search answered in a median of 50.7 ms, and the knowledge graph rebuilt in 143.3 seconds at 5,000 files after vectorisation. The original method took 246.1 seconds at only 2,000 files. Search slowed to 203.5 ms at 5,000 files, and the graph remains the main limit at that size. Two defects found during measurement, a name-resolution delay on the test machine and a quadratic graph computation, were fixed and re-measured. The report states what the measurements do and do not show, including the limits of the synthetic evaluation.

Keywords: semantic search, sentence embeddings, knowledge graph, local large language models, personal information management, privacy by design.

Abbreviations

Abbreviation	Expansion
API	Application Programming Interface
CSS / HTML	Cascading Style Sheets / HyperText Markup Language
DFD	Data Flow Diagram
ER	Entity-Relationship
HNSW	Hierarchical Navigable Small World (approximate nearest-neighbour index)
IICT	Institute of Information and Communication Technology
LLM	Large Language Model
MFT	Master File Table (NTFS file index)
NLI	Natural Language Interface
SQLite	Structured Query Language database engine (file-based)
UI / UX	User Interface / User Experience
UML	Unified Modeling Language

List of Figures

Figure	Title
4.1	System architecture
4.2	Module design
4.3	Data flow diagram - context level
4.4	Data flow diagram - level 1
4.5	Use case diagram
4.6	Class diagram of the response and settings data classes
4.7	Sequence diagram: open request
4.8	Sequence diagram: follow-up question
4.9	Entity-relationship diagram
5.1	Chat view
5.2	Knowledge graph view
5.3	Workspaces view

List of Tables

Table	Title
3.1	Functional requirements
3.2	Non-functional requirements
3.3	Hardware and software specification
4.1	Files table fields
4.2	Tables added for the graph, memory and workspace modules
6.1	Test levels and what each covered
6.2	Selected test cases and results
7.1	Scale benchmark: indexing and graph build
7.2	Scale benchmark: search latency and retrieval quality
7.3	Qualitative comparison with existing systems
7.4	Effect of the defect fixes on the measured pipeline

Chapter 1: Introduction

1.1 Introduction

Digital work leaves a trail of documents: invoices, résumés, meeting notes, code, photographs of receipts and downloads with names such as `final_final_v3.txt` or `Untitled document.txt`. Operating systems organise these by folder and filename, both chosen at the moment of saving. As the number of files grows, the names stop matching the way people remember them. A person looks for “the résumé with the internship section” and must recall an arbitrary name instead.

DreamOS addresses this by making meaning the access key. The user types a request such as “find my invoices”, “open my resume”, “what is related to my resume” or “suggest some workspaces”, and the system responds by searching, opening, relating or grouping files. The application is a desktop shell built with Tauri and React that talks to a local Python service. The service stores its index in two local databases and uses a local language-model runtime for understanding. Nothing is sent to a remote server.

1.2 Problem Statement

Design and implement a locally run file-management layer that lets a person retrieve, open, relate and organise files by describing what they are about, without depending on exact filenames, folder locations or a cloud service, and that makes its privacy behaviour verifiable.

1.3 Objectives

- Build a semantic index of a file collection using local embedding models, so that files are found by meaning.
- Provide one natural-language interface that classifies what the user wants and routes the request to the right module.
- Return ranked results with honest confidence, and open a file only when the match is confident.
- Use a local language model to summarise, tag and categorise files, and apply that organisation reversibly.
- Model relationships between files, remember conversation so that follow-up questions work, and recommend workspaces.

- Keep all processing local, and provide an audit that reports where data could leave the machine.
- Measure the system at a scale beyond the demonstration vault, and report the results including the defects found.

1.4 Scope of the Project

The project covers a single-user desktop application for Windows, indexing a folder of text, Markdown, source code, PDF and Word documents. It includes the six modules listed in Chapter 4, a privacy audit, and a scale evaluation on synthetic collections. It does not cover mobile devices, multi-user access, synchronisation across machines, cloud models, or indexing of images and audio. The demonstration uses a sandboxed vault of 37 generated files so that no real personal data is handled during evaluation.

Chapter 2: Literature Review

2.1 Review of Existing Systems and Related Work

Two lines of research bear on this project: dense text embeddings, which place text in a vector space where distance reflects meaning, and semantic or AI-native file systems, which use such embeddings to organise files. Mikolov et al. showed that words can be embedded so that geometric distance tracks relatedness [2]. Reimers and Gurevych extended the idea to sentences and paragraphs with Sentence-BERT, which makes comparing whole documents efficient [3]. DreamOS uses nomic-embed-text, a model in this family, served locally through Ollama, with the Chroma vector database for nearest-neighbour search [4].

The closest system is LSFS, a semantic file system for an LLM operating-system agent presented at ICLR 2025 [1]. LSFS shows that embedding indexes plus a language-model front end can replace command-based file operations. It is designed for software agents that call an API, not for a person typing requests, and its evaluation does not establish that it runs entirely on local models. DreamOS takes the same architectural idea and exposes it to a human through one conversational surface that both finds and acts on files.

Mainstream desktop search is still driven by names and metadata. Windows Search indexes filenames, metadata and text but matches a query only where it shares vocabulary with the file [5]. Spotlight adds metadata and heuristics over a similar keyword index [6]. Everything indexes the NTFS master file table for instant filename lookup and does not read content [7].

Table 2.1: Comparison with existing systems

System	Retrieval basis	Strength	Limitation relative to DreamOS
Windows Search	Keyword and metadata index	Built in, fast	No meaning-based matching; cannot act on a result from one query
macOS Spotlight	Keyword, metadata, heuristics	Fast, system-wide	Not embedding-based; not conversational
Everything	Filename index (MFT)	Very fast name lookup	Names only; no content understanding
LSFS (ICLR 2025)	LLM plus embedding index	Shows embedding file indexing at OS level	Built for agents via API; local operation not demonstrated
DreamOS	Local embeddings plus local intent routing	One interface to find, open, relate and organise; private	Single-user desktop only; small-model limits on short queries

2.2 Research Gap

The literature establishes embedding-based file indexing as workable [1] and establishes that consumer tools match on words [5-7]. Three gaps remain. First, embedding search is presented as an agent API rather than as a single conversational surface that both retrieves and acts on files for a person. Second, published systems do not show that the full pipeline, from embedding to answer, runs on local models with the data-leaving question answered by measurement rather than assertion. Third, the behaviour of such systems on short, everyday queries, and on collections of thousands of files, is rarely reported with the defects that testing exposes.

2.3 Summary of Findings

- Embedding similarity is compressed for sentence models, so fixed similarity cut-offs drop correct answers to short queries; a relative cut-off with a small keyword boost recovers them without re-embedding.
- A language model's own judgement about whether a message refers back to earlier conversation is unreliable; deterministic rules and direct evidence should decide first.
- Pairwise similarity over a whole collection grows with the square of the number of files, and must be vectorised before it can scale.
- Averaging edge weights, rather than merging connected components, prevents unrelated groups from being joined by a single weak link.

Chapter 3: System Analysis

3.1 Requirement Analysis

3.1.1 Functional requirements

Table 3.1: Functional requirements

ID	Requirement	Status
FR1	Accept one natural-language message through a chat interface	Implemented
FR2	Classify intent as search, open, related, workspace, organise or other	Implemented
FR3	Semantic search returning ranked files with score and snippet	Implemented
FR4	Open the best match in its default application only when confident	Implemented
FR5	Generate a summary, tags and category for each file	Implemented
FR6	Apply and reverse an organisation move	Implemented
FR7	Preview unorganised files before any change	Implemented
FR8	Index and re-index incrementally by content hash	Implemented
FR9	Model relationships between files and show related files	Implemented
FR10	Resolve follow-up questions against the earlier conversation	Implemented
FR11	Recommend and create workspaces from relationships and usage	Implemented
FR12	Report where data could leave the machine, from live configuration	Implemented

3.1.2 Non-functional requirements

Table 3.2: Non-functional requirements

Attribute	Requirement	How it is evaluated
Privacy	No file content or question leaves the machine	Privacy audit; loopback check; test
Performance	Search answers in well under a second	Scale benchmark, Section 7
Scalability	Graph and indexing remain practical at several thousand files	Scale benchmark at 500, 2,000 and 5,000 files
Reliability	Organisation moves are reversible	Automated apply-and-revert test
Usability	One input box; results are cards with actions	Screenshots, Section 5.4
Maintainability	Each module has its own tests; offline test run	116 automated tests, CI on every push

3.2 Feasibility Study

Technical feasibility was confirmed early by building the indexing and search path with a local embedding model and measuring it on the demonstration vault. Economic feasibility holds because every component is open source and the models run on an ordinary laptop without a graphics card, so the running cost is zero. Operational feasibility depends on the user accepting that the first request after a cold start is slow while a model loads, which is documented in the demonstration guide. Schedule feasibility was managed by building the three core modules first and then adding the graph, memory and workspace modules as separate, tested increments.

3.3 System Specifications

Table 3.3: Hardware and software specification

Item	Specification used for development and evaluation
Processor	AMD Ryzen 7 6800H, 8 cores, 16 logical processors
Memory	16 GB installed (about 15 GB reported as usable)
Storage	Local solid-state storage; index size is reported per run in the benchmark output
Graphics	Not required; models run on the CPU
Operating system	Windows 11
Language and frameworks	Python 3.12, FastAPI, Pydantic, ChromaDB 1.5, SQLite 3, NumPy 2.5
Models	nomic-embed-text (137M parameters, embeddings); llama3.2 3B (intent, summaries, tags)
Desktop shell	Tauri 2 with React 19, TypeScript and Vite
Testing	pytest 9, FastAPI test client, GitHub Actions on Linux
Packaging	Windows MSI and NSIS installers built by the Tauri bundler

Chapter 4: System Design

4.1 System Architecture

DreamOS is a client-server application on one machine. The desktop client calls a local service over HTTP on 127.0.0.1:8420. The service classifies requests, searches, organises and computes relationships, and calls the local model runtime for language tasks. Two stores hold the data: a relational database for metadata, relationships, conversation and usage, and a vector database for the embeddings of text chunks.

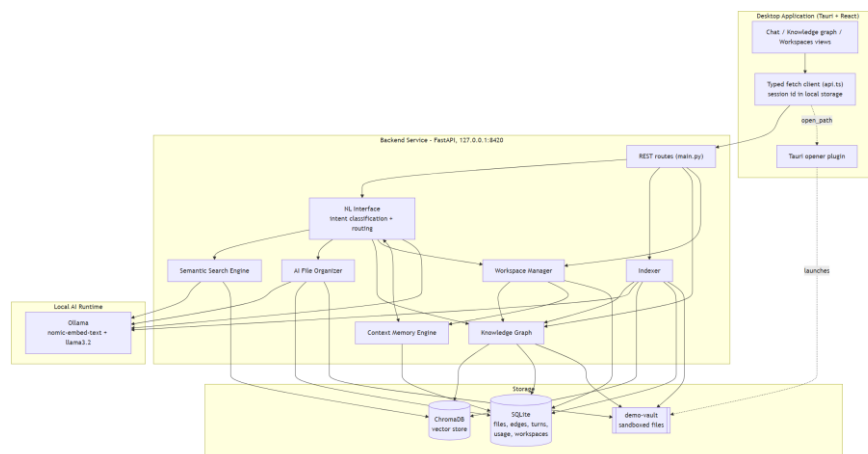


Figure 4.1 - System architecture: desktop client, local service, local model runtime, and storage

4.2 Module Design

The system is organised in six modules. The Natural Language Interface decides what a request means and routes it. The Semantic Search Engine returns files near the best match. The AI File Organizer proposes categories and reversible moves. The Knowledge Graph links files by similar content, by direct reference and by shared tags. The Context Memory Engine keeps the conversation so that a follow-up can be resolved, and records which files the user opens. The Workspace Manager turns the graph and usage into explainable recommendations and creates workspaces that open together.

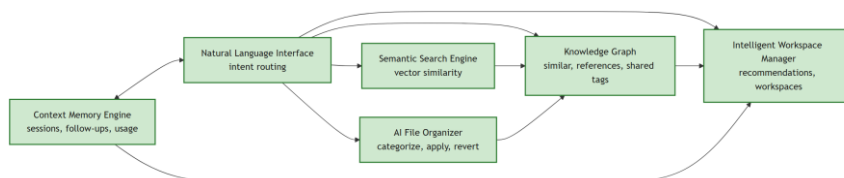


Figure 4.2 - Module design and the data each module passes to the next

4.3 Data Flow Diagrams

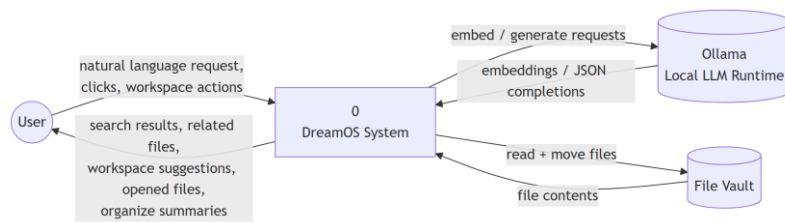


Figure 4.3 - Data flow diagram, context level

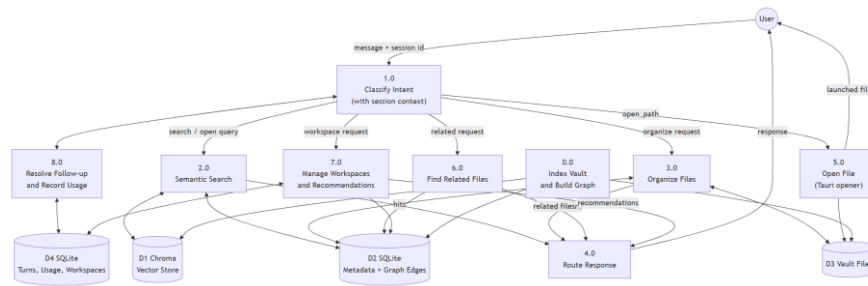


Figure 4.4 - Data flow diagram, level 1

4.4 UML Diagrams

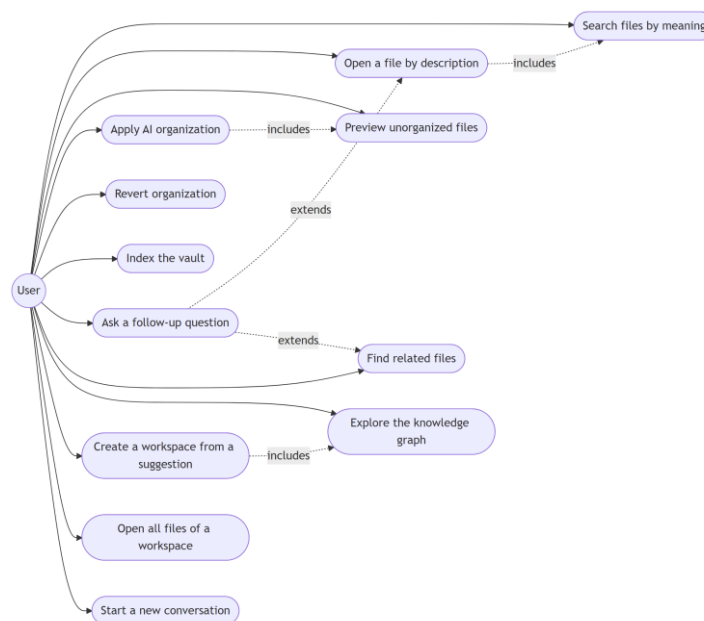


Figure 4.5 - Use case diagram of user actions

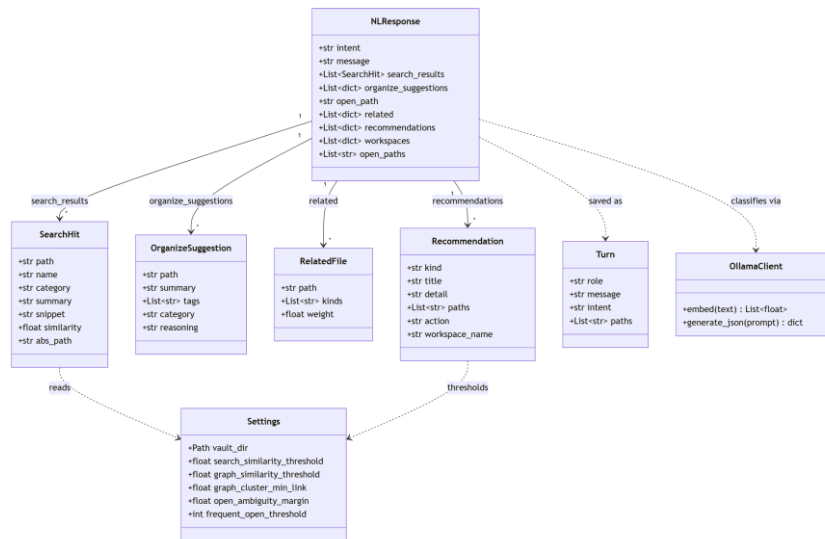


Figure 4.6 - Class diagram of response and settings classes

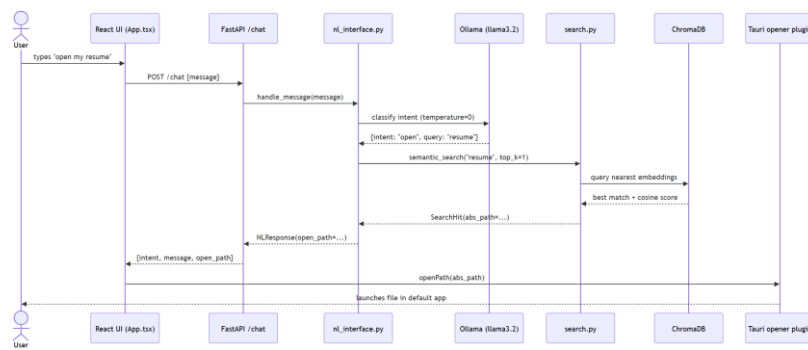


Figure 4.7 - Sequence diagram of an open request

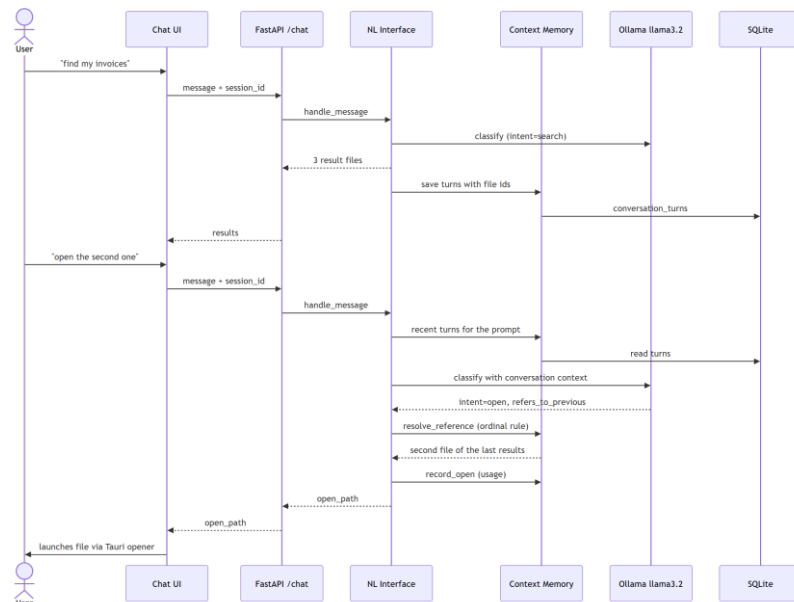


Figure 4.8 - Sequence diagram of a follow-up question resolved from memory

4.5 Database Design

File metadata is stored in SQLite and each file's text chunks are stored as vectors in ChromaDB, linked by the file path. All relationship, memory, usage and workspace tables refer to the file's numeric identifier rather than its path. The organiser renames a file when it moves it, but the identifier does not change, so no table needs rewriting after a move.

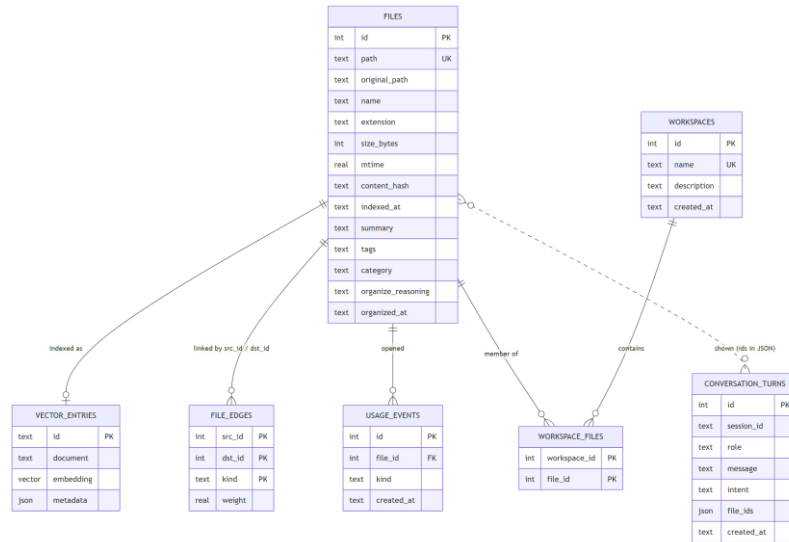


Figure 4.9 - Entity-relationship diagram

Table 4.1: Fields of the files table

Field	Type	Purpose
id	INTEGER	Stable identifier used by every other table
path	TEXT, unique	Path relative to the vault; also the vector identifier
original_path	TEXT	Path at first indexing, kept so a move can be reversed
content_hash	TEXT	Lets re-indexing skip unchanged files
summary, tags, category	TEXT	Produced by the organiser
organize_reasoning, organized_at	TEXT	Explanation and time of the applied organisation

Table 4.2: Tables added for the graph, memory and workspace modules

Table	Key fields	Purpose
file_edges	source, target, kind, weight	Typed relationships between files
conversation_turns	session, role, message, file identifiers	Conversation kept across restarts
usage_events	file identifier, kind, time	Records each open; used for tie-breaking and recommendations
workspaces, workspace_files	name; workspace and file identifiers	Named groups of files

4.6 UI / UX Design

The interface has four views: a chat view for requests, a knowledge-graph view, a workspace view and a privacy view. Results appear as cards with the action the user can take, so a file is opened from the card that shows it. Related-file answers show why each file is linked, such as similar content, a direct reference or shared tags.

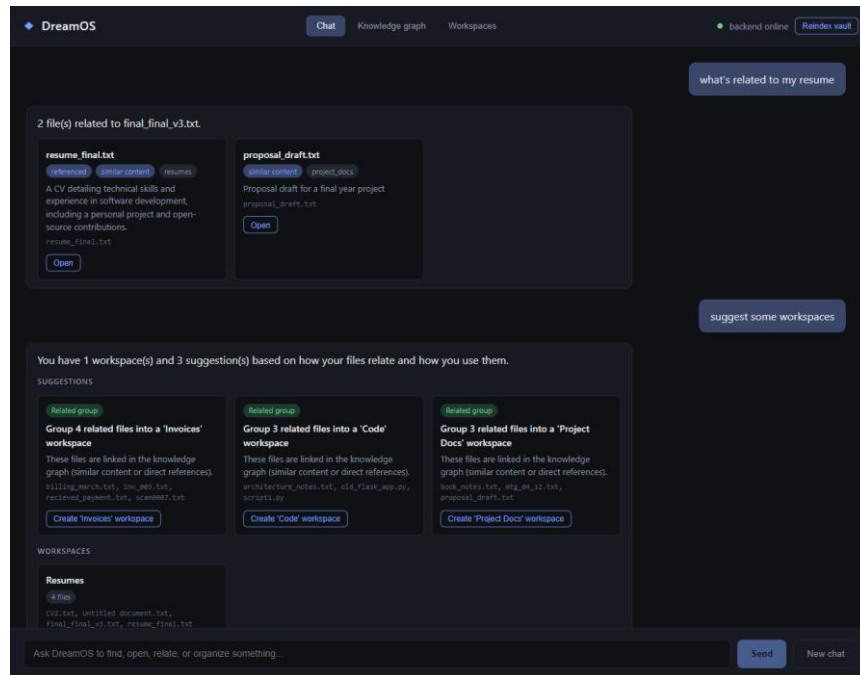


Figure 4.10 - Chat view: related files with the reason each is linked

4.7 Technology Stack

Table 4.3: Technology stack by layer

Layer	Technology
Interface	React 19, TypeScript, Vite, d3-force for the graph layout, Tauri 2 shell
Service	Python 3.12, FastAPI, Uvicorn, Pydantic
Understanding	Ollama runtime with nomic-embed-text and llama3.2
Storage	SQLite for relational data; ChromaDB for vectors
Numerics	NumPy for vectorised similarity
Quality	pytest, GitHub Actions, Playwright and the WebView2 debug protocol for end-to-end checks

Chapter 5: Implementation

5.1 Module-wise Description

5.1.1 Natural Language Interface

Each message is sent to the local model with a short description of the four levels of intent, and the model returns JSON with the intent, the topic to search for, and a flag for whether the message refers back to earlier conversation. Requests are routed deterministically after that. The interface reports a low-confidence result as a guess and never launches a file on a weak match.

5.1.2 Semantic Search Engine

A query is embedded and compared with every chunk. Results are grouped by file. A file is kept if it lies within a margin of the best hit and above a floor, and a small boost is added where the query's words appear in the file's text or name. This replaced a fixed cut-off, which had returned nothing for short queries such as “meeting”.

5.1.3 AI File Organiser

For each file the model proposes a summary, tags, a category and the reason for the category. The user applies or refuses each proposal. Applying moves the file into a folder named after the category and records the original path. Reverting moves it back and removes the folder if it is empty.

5.1.4 Knowledge Graph

Three kinds of edge are computed: similar content, from the cosine similarity of the centred mean embedding of each file; direct reference, when one file's text names another file; and shared tags, when two files share at least two tags. Each file keeps its three strongest neighbours. Clusters are built by average-linkage merging, which joins two groups only while the average weight between their members stays above a threshold.

5.1.5 Context Memory Engine

Every turn is stored with the identifiers of the files it showed. A follow-up such as “open the second one” or “open it” resolves to a file by rule. Only when no rule applies, and no confident search matches the request, is the model's own judgement used. This ordering was introduced after a live failure described in Section 7.3.

5.1.6 Intelligent Workspace Manager

Recommendations come from rules, not from the language model, so they are reproducible and explainable. They include related groups, frequently opened files, possible duplicates, files without a category and stale files. A workspace is a named group whose files open together.

5.1.7 Privacy audit

The audit reads the live configuration and lists each network endpoint the service uses, whether it is on this computer, what it is used for, and what is stored locally. It says plainly that it checks configuration, not network traffic, and shows how to confirm the listening address with netstat.

5.2 Technologies Used

The service is written in Python with FastAPI, which provides typed request models and an automatic test client. ChromaDB stores vectors on disk, and SQLite stores everything relational. NumPy performs the pairwise similarity computation as a single matrix product. The desktop shell is Tauri, which hosts a React interface and runs the operating-system file opener under an explicit permission scope. The language model runtime is Ollama, which serves both models over a local HTTP interface.

5.3 Code Snippets

The following excerpt is the vectorised similarity computation. One matrix product gives every pairwise cosine similarity, and the strongest neighbours of each file are selected in one pass. The original version compared each pair in a Python loop.

```
unit = matrix / norms[:, None]
sims = unit @ unit.T
np.fill_diagonal(sims, -np.inf)
k = min(settings.graph_top_k, len(paths) - 1)
neighbours = np.argpartition(-sims, k - 1, axis=1)[: , :k]
```

The excerpt below is the rule that decides whether a follow-up refers to earlier results. Deterministic evidence is checked first, and the model's flag is only a last resort.

```
def _open(query, referenced, weak_referenced=None):
    if referenced:
        # rule-based: ordinal or bare "it"
        return _open_response(search.hit_for_path(referenced))
    hits = search.semantic_search(query, top_k=3)
    best = hits[0] if hits else None
    if not best or best.similarity < settings.search_similarity_threshold:
        if weak_referenced:
            # model's guess, last resort only
            return _open_response(search.hit_for_path(weak_referenced))
```

5.4 Screenshots of Outputs

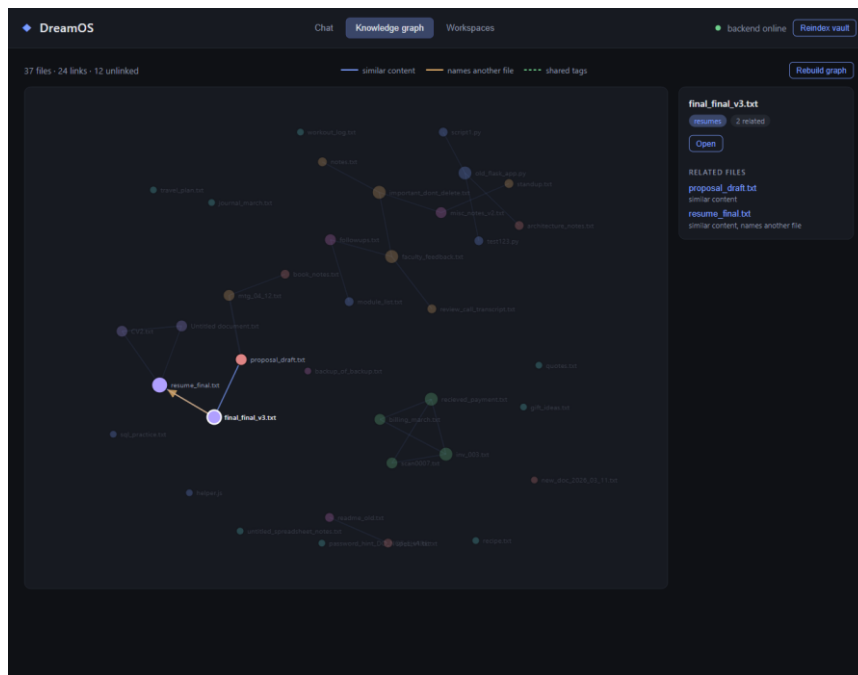


Figure 5.1 - Knowledge-graph view with a selected file and its neighbours

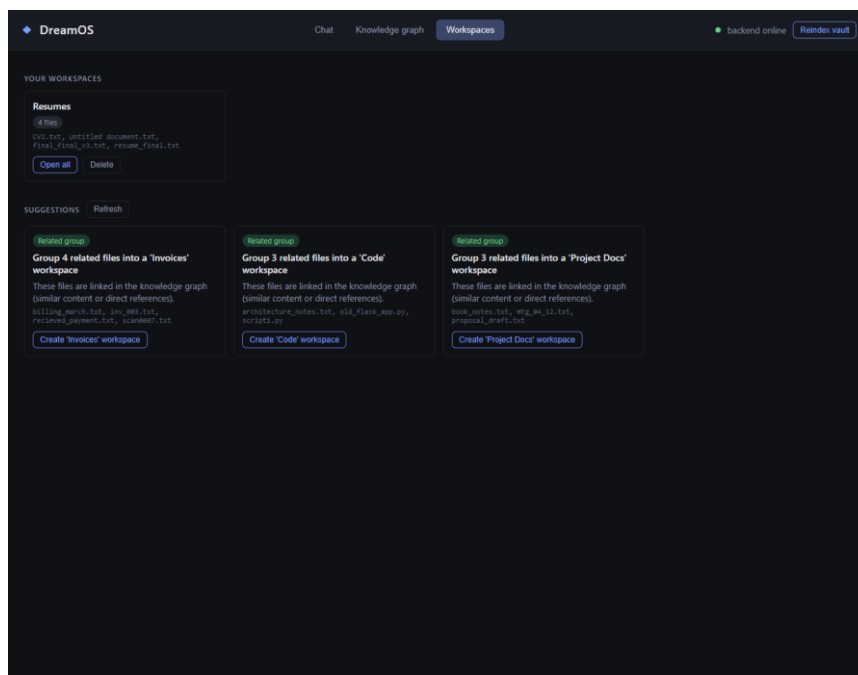


Figure 5.2 - Workspace view with a created workspace and suggestions

Chapter 6: Testing and Validation

6.1 Testing Types

Table 6.1: Test levels and what each covered

Level	What was tested	Where
Unit	Cosine and centring, cut-off and boost, reference detection, cluster merging, ordinals, memory rules	tests/test_*.py
Integration	Indexing to search, organise and revert across the database and vector store; intent routing with the memory	tests/test_nl_interface.py, test_indexer.py
API	Every HTTP route through the FastAPI test client, including error codes	tests/test_api.py
Equivalence	The vectorised similarity against the original pairwise loop on random data	tests/test_knowledge_graph.py
System	Live runs against the real model runtime and vault; desktop shell driven over the WebView2 debug protocol	Sections 6.2 and 7.1

6.2 Test Cases and Results

The automated suite runs offline: model calls are replaced by fixtures that return fixed vectors, so the suite needs no model runtime and runs in well under a minute. It runs on every push in continuous integration. The table shows selected cases; each one was also run against the live models.

Table 6.2: Selected test cases and results

Test case	Expected	Result
Find a file by meaning, not name	Relevant file ranked first	Passed; “invoice from the hosting vendor” returns the vendor invoice
Short query with a weak but correct best hit	Returned, labelled as low confidence	Passed
Open a weak match	Not launched; candidates shown	Passed
Open “the second one” after a search	Second result opened	Passed
Follow-up on a new topic after an old one	New topic wins over stale context	Passed after fix (Section 7.3)
Apply then revert an organisation	File returns to original place; empty folder removed	Passed
Relationships survive a file move	Edges still attached to the moved file	Passed

Test case	Expected	Result
Vectorised similarity equals original loop	Identical edges on random data	Passed
Remote model endpoint reported	Marked as leaving the machine	Passed
Workspace create then open all	Every file opened	Passed in the desktop shell

Chapter 7: Results and Discussion

7.1 Performance Evaluation

Scale was measured on synthetic collections of 500, 2,000 and 5,000 files. The collections were generated across twelve topics, such as invoices, recipes and research notes, and about half of the documents include one sentence from an unrelated topic, so that the collection is not trivially separable. Each run used a fresh database and measured the real pipeline with the real models: embedding through Ollama, storage in ChromaDB and SQLite, graph construction, and 36 search queries with their latency. The benchmark script is part of the repository (backend/scripts/bench_scale.py) so that the measurements can be reproduced.

Table 7.1: Scale benchmark - indexing and knowledge-graph build (after the fixes)

Files	Index time (s)	Index rate (files/s)	Graph rebuild (s)	Graph links
500	25.7	19.4	1.7	1062
2,000	129.0	15.5	24.8	4747
5,000	446.5	11.2	143.3	11938

Index time includes the knowledge-graph rebuild, which runs after each indexing pass that changed something. Before the vectorisation the same measurement at 2,000 files was 372.5 seconds for indexing, including a graph rebuild of 246.1 seconds. The 5,000-file graph rebuild of 143.3 seconds shows that the pairwise reference check and the clustering step still grow faster than linearly; the report treats this as a limitation rather than a solved problem.

Table 7.2: Search latency and retrieval quality

Files	p50 latency (ms)	p95 latency (ms)	Precision@5	Queries with no result
500	50.7	61.3	1.00	0
2,000	121.0	140.3	1.00	0
5,000	203.5	239.0	1.00	0

Precision@5 is the share of the five best results that come from the topic the query describes. On this synthetic data it stays at the maximum, so it shows that the topics separate, but it does not measure retrieval on real documents. A harder evaluation on real files is listed as future work.

7.2 Comparison with Existing Systems

Table 7.3 compares the design with the systems reviewed in Chapter 2. The comparison is qualitative: the other systems were not run in this evaluation, so no numerical comparison is claimed.

Table 7.3: Qualitative comparison with existing systems

Capability	Windows Search / Spotlight	LSFS	DreamOS
Matches by meaning	No	Yes	Yes
Single conversational interface for a person	Search box only	No (agent API)	Yes
Acts on results (open, organise, reverse)	Open only	Yes, for agents	Yes, with confirmation
Runs entirely on local models	Not applicable	Not shown	Yes; checked by audit
Verifiable privacy report	No	No	Yes (configuration level)

7.3 Observations

Table 7.4: Effect of the defect fixes on the measured pipeline

Measure	Before fix	After fix	Change
Embedding call latency on the test machine	about 1,000 ms (name resolution)	about 15 ms	about 65 times faster
Indexing, 100-file test (files/s)	0.41	19.6	about 48 times faster
Indexing, 500 files (files/s, includes graph)	12.4	19.4	about 1.6 times faster
Graph rebuild at 2,000 files (s)	246.1	24.8	about 10 times faster
Graph rebuild at 5,000 files (s)	not measured (projected to be hours)	143.3	now measurable; still the main limit

Three findings from the work matter beyond the code itself. First, a delay of about one second on every model call was caused by the way the machine resolved the name localhost, which tries an IPv6 address first. The fix was to use the IPv4 loopback address directly. Without measurement this would have been blamed on the model. Second, the graph computation was quadratic in the number of files: at 2,000 files it took about fourteen times as long as at 500 files for four times the files, and it would have been unusable at the scale required. Vectorisation, with an equivalence test against the original loop, resolved it. Third, a live rehearsal exposed a failure that the automated tests had not covered: a follow-up question about

a new topic was answered with an old result, because the model judged the sentence to refer back to the conversation. The fix gives direct evidence precedence over the model's judgement, and a regression test was written that fails on the old behaviour.

Retrieval on short queries remains the weakest part. A one-word query can rank a file that merely contains the word above the file the user meant. The system reports such results with a confidence label instead of acting on them, which keeps the user in control.

Chapter 8: Conclusion and Future Scope

8.1 Summary of Work

DreamOS was built as a local, meaning-based file manager with six modules, a desktop interface and a privacy audit. It indexes a folder with local embeddings, answers natural-language requests, opens files only on confident matches, organises files reversibly, relates files through a knowledge graph, remembers conversation so that follow-ups work, and recommends workspaces. It was measured on collections of up to 5,000 files, and the measurements led to two defect fixes and one behavioural fix, each covered by a test.

8.2 Limitations

- The scale evaluation uses synthetic documents. Retrieval quality on real personal files has not been measured.
- Short, generic queries can rank a file that contains the word above the intended file.
- The 3-billion-parameter model misclassifies some borderline wording, for example routing “which files should I archive” to organise rather than workspace.
- The graph rebuild still grows faster than linearly: 143 seconds at 5,000 files. Very large collections would need an approximate nearest-neighbour index and a cheaper reference check.
- The privacy audit checks configuration; it does not capture network traffic. The report describes the netstat check that does.
- The system runs on Windows only and handles text, Markdown, source, PDF and Word files.

8.3 Future Enhancement

- Evaluate retrieval on a labelled set of real documents, with the participants' consent and without storing their content.
- Replace the pairwise graph with an approximate nearest-neighbour index for collections above 10,000 files.
- Test the embedding task prefixes end to end, now that a labelled evaluation exists to judge them.
- Add an image and scanned-document pipeline with local optical character recognition.

- Package the model runtime with the installer so that first use does not require a separate download.

References

- [1] Z. Shi et al., “From Commands to Prompts: LLM-based Semantic File System for AIOS,” in Proc. ICLR 2025, arXiv:2410.11843.
- [2] T. Mikolov, K. Chen, G. Corrado and J. Dean, “Efficient Estimation of Word Representations in Vector Space,” arXiv:1301.3781, 2013.
- [3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in Proc. EMNLP-IJCNLP, 2019.
- [4] Chroma, “Chroma: the open-source embedding database,” github.com/chroma-core/chroma.
- [5] Microsoft, “Windows Search overview,” Microsoft Learn documentation.
- [6] Apple Inc., “Spotlight User Guide,” support.apple.com.
- [7] voidtools, “Everything - locate files and folders by name instantly,” voidtools.com.
- [8] Ollama, “Ollama: run large language models locally,” ollama.com.

Appendices

Appendix A: Source code

The complete source code is in the private repository github.com/zaidwhy/dreamos-college-project, which is available to the examiners on request. The main files are backend/app (service), frontend/src (interface), backend/tests (automated tests) and backend/scripts/bench_scale.py (scale benchmark).

Appendix B: Scale benchmark raw results

Table B.1: Raw benchmark output at 500, 2,000 and 5,000 files

Measure	500 files	2,000 files	5,000 files
Files indexed	500	2000	5000
Index time (s)	25.7	129.0	446.5
Graph rebuild (s)	1.7	24.8	143.3
Graph links	1062	4747	11938
Search p50 / p95 (ms)	50.7 / 61.3	121.0 / 140.3	203.5 / 239.0

Appendix C: Papers and certificates

No research paper has been accepted or presented as part of this project at the time of writing. Acceptance letters and presentation certificates, if any are issued, will be attached here.